

Chương 2. Công nghệ J2EE và Lập trình Servlet/JSP

2.1. Giới thiệu Servlet/JSP

- ▶ Giải thích mô hình công nghệ web
- ▶ Mô tả công nghệ Servlet/JSP
- ▶ Cài đặt môi trường phát triển
- ▶ Tạo dự án web động
- ▶ Tổ chức dự án theo mô hình MVC
- ▶ Đóng gói và triển khai ứng dụng web.

Giới thiệu

- ▶ Thế giới web
- ▶ Mô hình ứng dụng web
- ▶ Mô hình công nghệ 3 lớp
- ▶ Mô hình công nghệ Servlet/JSP
- ▶ Cài đặt môi trường phát triển
- ▶ Tạo dự án web động và chạy thử
- ▶ Tổ chức dự án web động
- ▶ Tổ chức mô hình MVC
- ▶ Truyền dữ liệu từ Servlet sang JSP
- ▶ Làm việc với tham số đơn giản
- ▶ Đóng gói và triển khai

Giới thiệu thế giới web

4

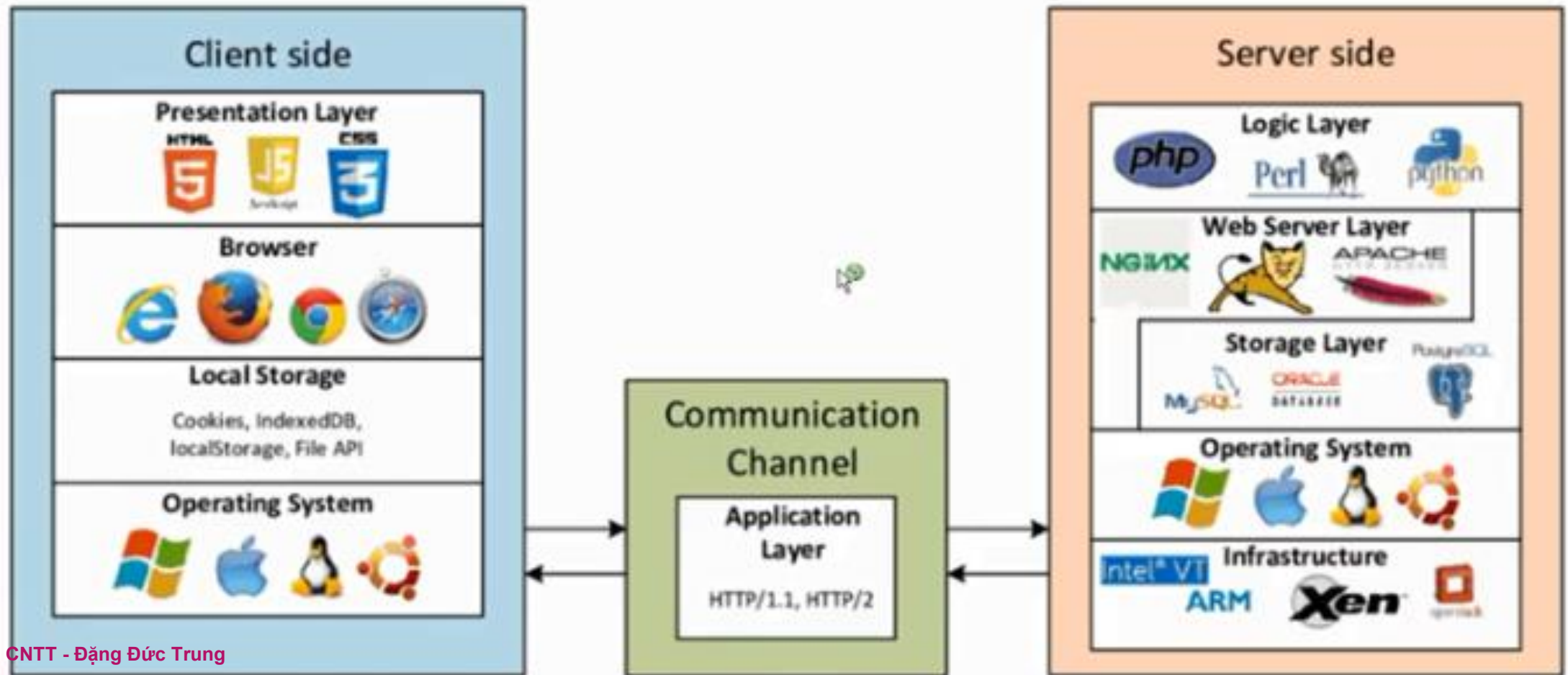
- ▶ Các dịch vụ và ứng dụng trên môi trường Internet



Giới thiệu Internet và web



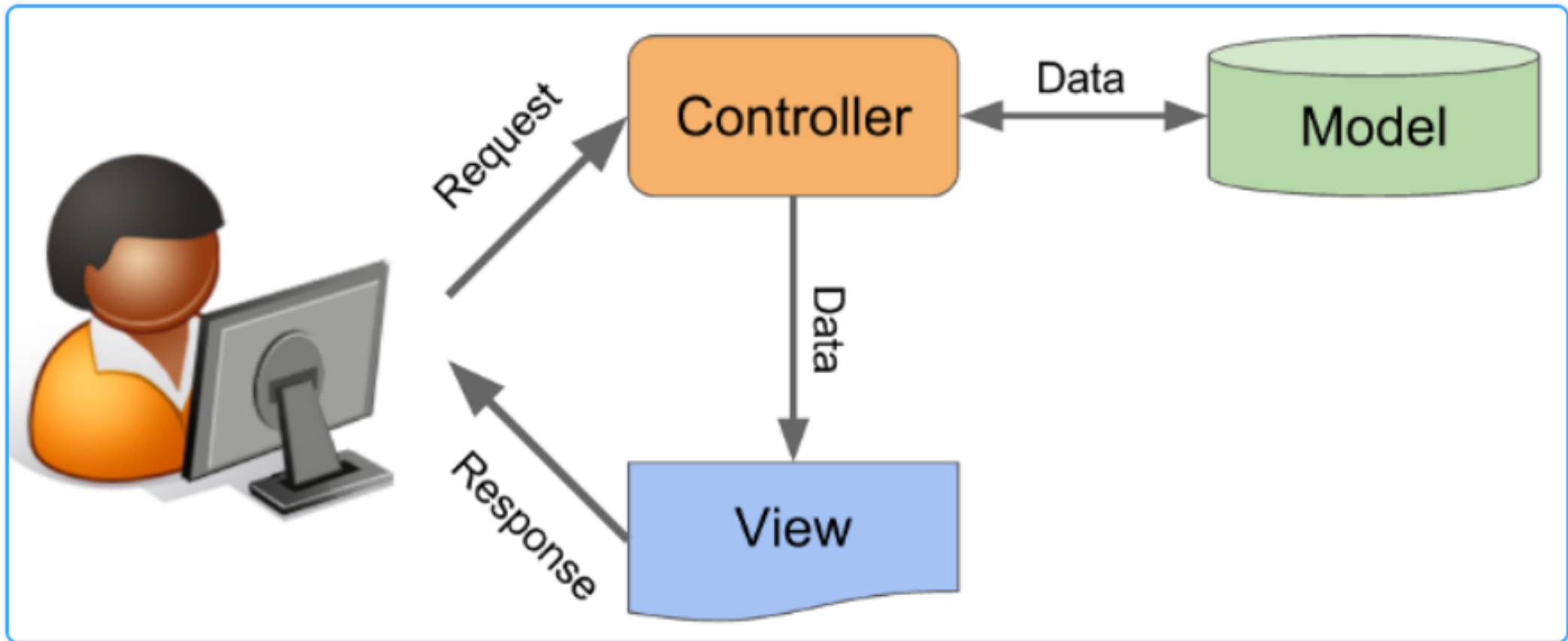
Giới thiệu công nghệ web



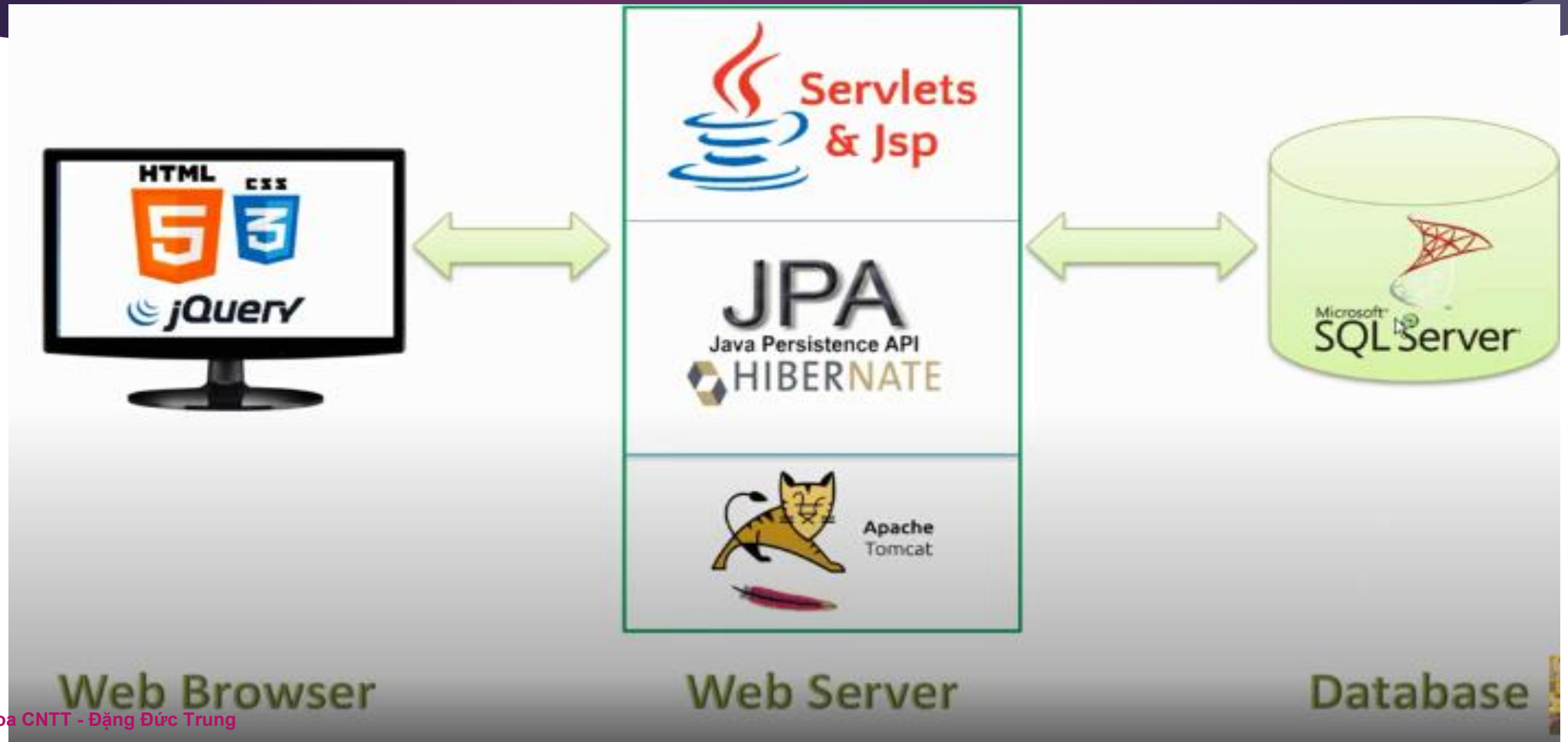
Giới thiệu mô hình 3 lớp



Giới thiệu mô hình MVC



Giới thiệu công nghệ áp dụng cho môn học

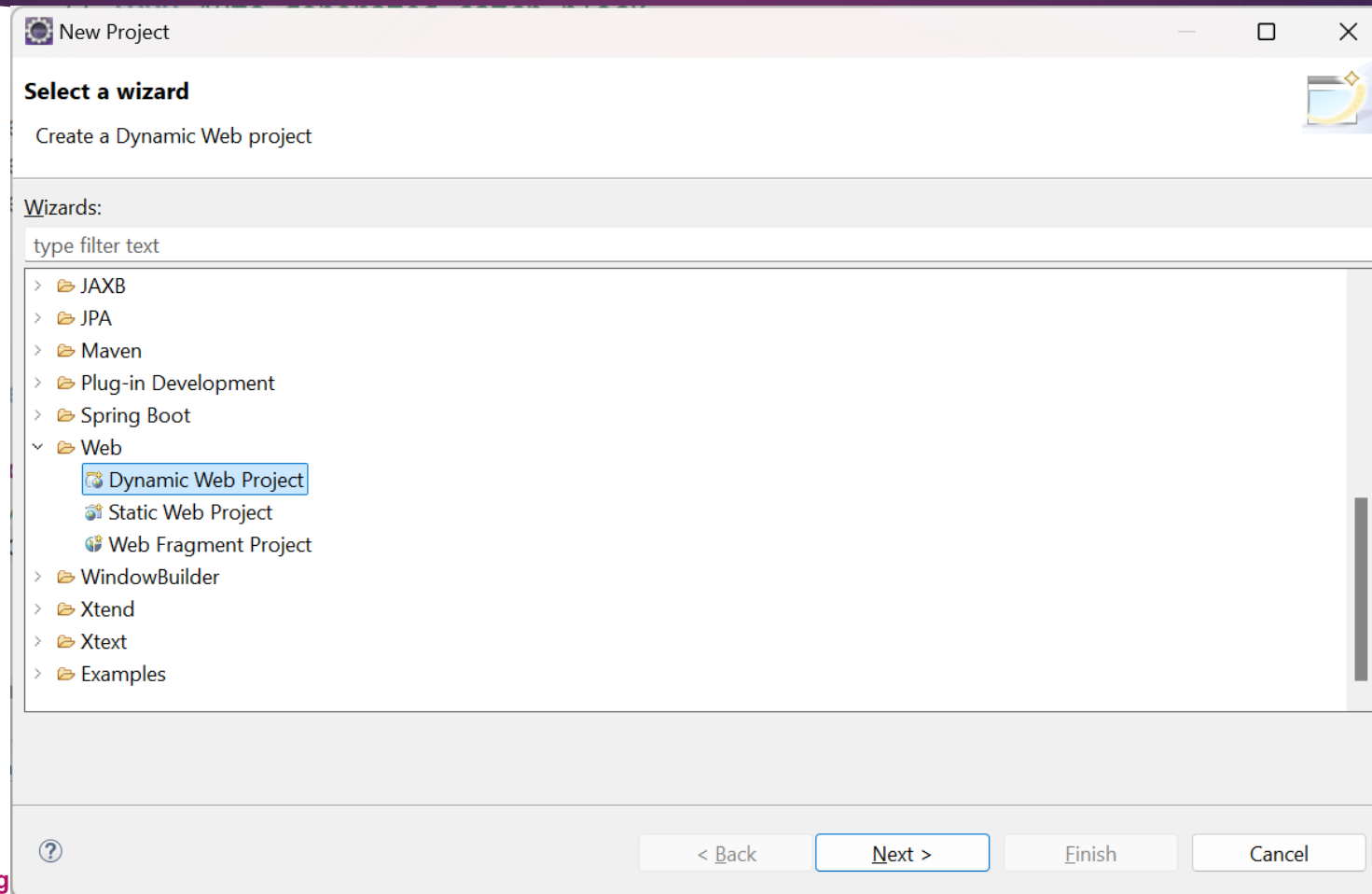


Giới thiệu Tomcat và Eclipse

- ▶ Tomcat là webserver, Eclipse for EE là công cụ phát triển ứng dụng web. Ứng dụng web sau khi phát triển cần đóng gói và triển khai vào Tomcat mới có thể chạy được.
- ▶ Eclipse for EE cho phép tích hợp Tomcat vào trong Web project khi tạo mới project. Không cần phải đóng gói ứng dụng vẫn có thể chạy ứng dụng ngay khi đang phát triển.
- ▶ Tích hợp sẽ giúp chúng ta:
 - ▶ Dễ dàng trong debug
 - ▶ Đơn giản việc test nhanh ứng dụng.

Tạo một dự án web động/web app mới 11

- File / New Project/ Dynamic Web Project



Đặt tên dự án và chọn phiên bản Server Tomcat

12

New Dynamic Web Project

Create a standalone Java-based Web Application or add it to a new or existing Enterprise Application.

Project name:

Project location

☒ Use default location

Location:

Target runtime

Dynamic web module version

Configuration

The default configuration provides a good starting point. Additional facets can later be installed to add new functionality to the project.

EAR membership

☐ Add project to an EAR

EAR project name:

Working sets

☐ Add project to working sets

Working sets:

New Server Runtime Environment

Define a new server runtime environment

Select the type of runtime environment:

- Apache
 - Apache Tomcat v7.0
 - Apache Tomcat v8.0
 - Apache Tomcat v8.5
 - Apache Tomcat v9.0
 - Apache Tomcat v10.0
 - Apache Tomcat v10.1

Chỉ định tên server Tomcat và thư mục gốc của Tomcat

New Server Runtime Environment

Tomcat Server

Specify the Tomcat installation directory and JRE for this runtime. The specified JRE controls the highest supported Java Facet version.

Name:
Apache Tomcat v8.5(1)

Tomcat installation directory:
C:\apache-tomcat-8.5.34 Browse...

apache-tomcat-8.5.99 Download and Install...

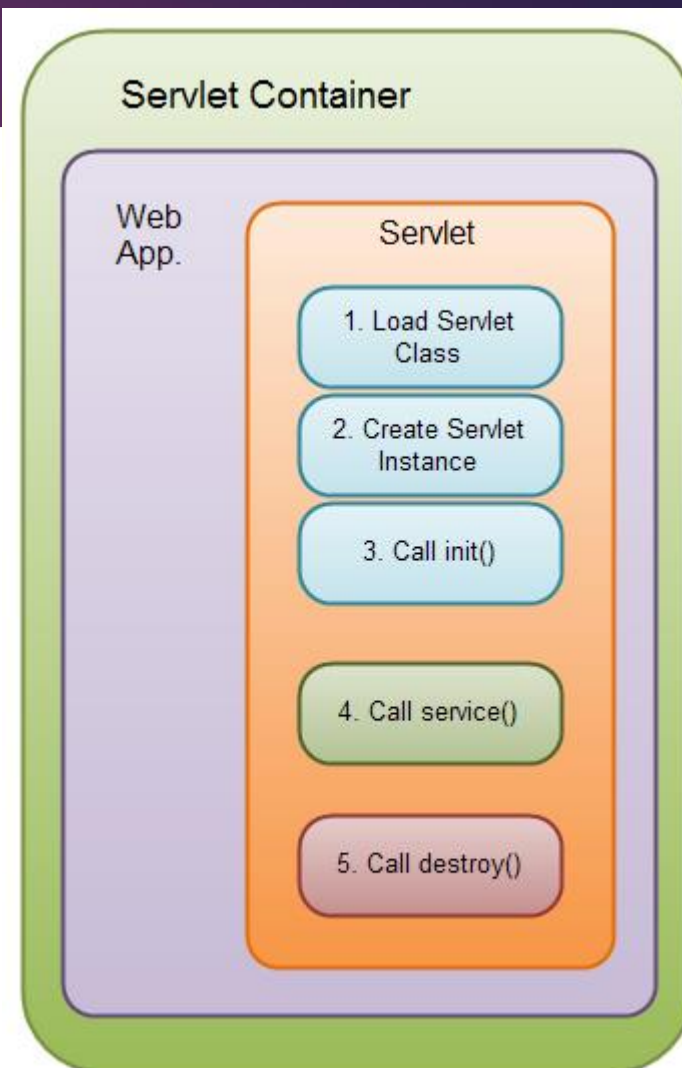
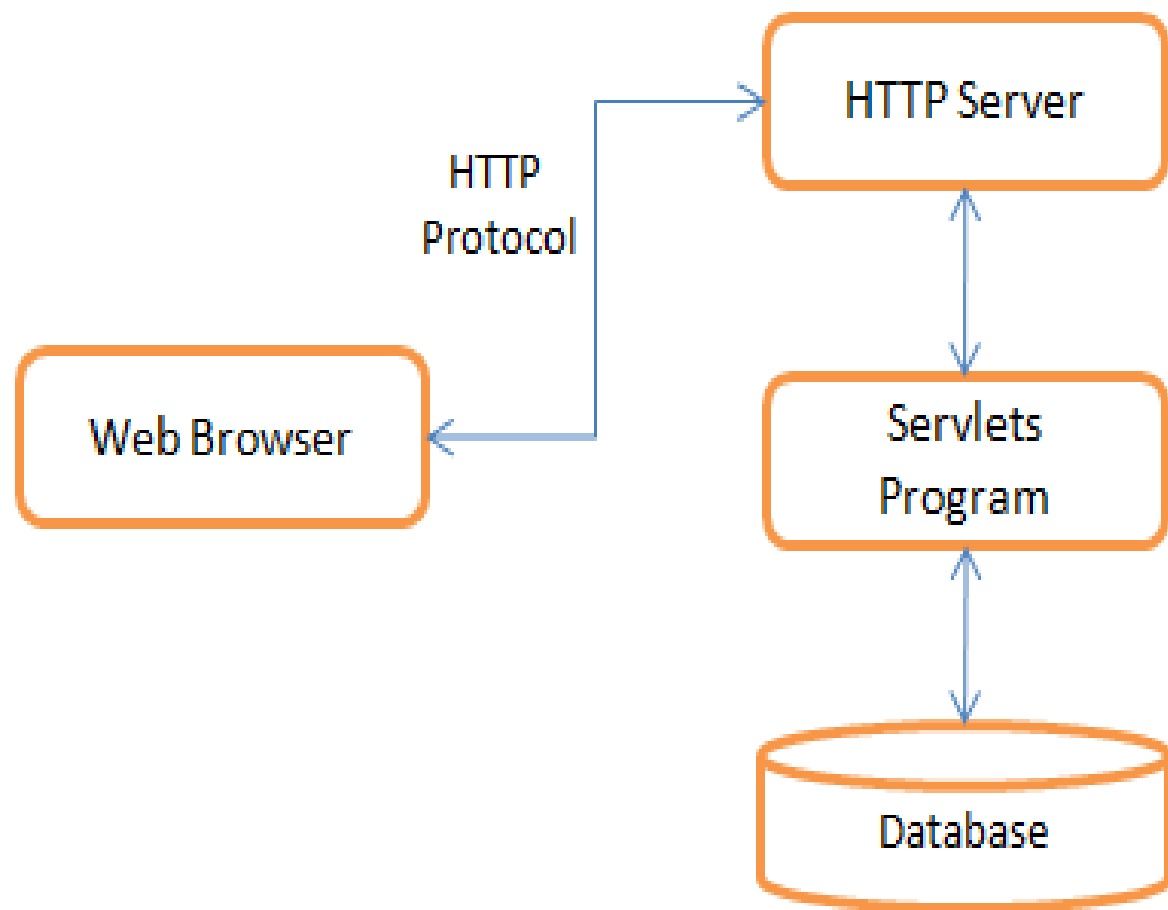
JRE:
Workbench default JRE Installed JREs...

? < Back Next > Finish Cancel

Giới thiệu Servlet

- ▶ Java Servlet là chương trình chạy trên một web hoặc ứng dụng máy chủ (Application Server) và hành động như một lớp trung gian giữa một yêu cầu đến từ trình duyệt web hoặc HTTP khách (client) khác và cơ sở dữ liệu hoặc các ứng dụng trên máy chủ HTTP (HTTP server)
- ▶ Servlet là một công nghệ được sử dụng để tạo ra ứng dụng web.
- ▶ Servlet là một API cung cấp các interface và lớp bao gồm các tài liệu.
- ▶ Servlet là một thành phần web được triển khai trên máy chủ để tạo ra trang web động.

Vị trí và vòng đời của Servlet



Bên trong một servlet có gì?

16

```
package Servlets;
```

```
+ import ...7 lines
```

```
@WebServlet(name = "HelloServlet", urlPatterns = {"/HelloServlet"})
```

```
public class HelloServlet extends HttpServlet {
```

```
    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
```

```
    throws ServletException, IOException {
```

```
        response.setContentType("text/html;charset=UTF-8");
```

```
        try (PrintWriter out = response.getWriter()) {
```

```
            /* TODO output your page here. You may use
```

```
            out.println("<!DOCTYPE html>");
```

```
            out.println("<html>");
```

```
            out.println("<head>");
```

```
            out.println("<title>Servlet HelloServlet</t
```

```
            out.println("</head>");
```

```
            out.println("<body>");
```

```
            out.println("<h1>Servlet HelloServlet at "
```

```
            out.println("<p>Request URI:" + request.getR
```

```
            out.println("<p>Protocol:" + request.getProt
```

```
            out.println("<p>Path Info:" + request.getPat
```

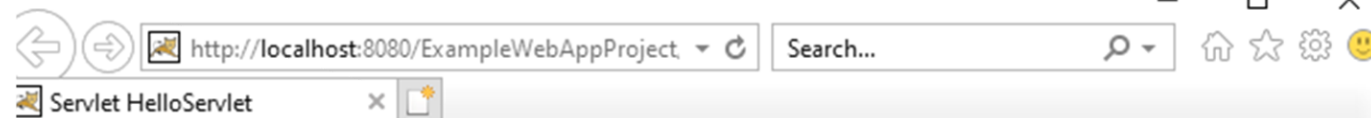
```
            out.println("<p>A random number:<strong>" +
```

```
            out.println("</body>");
```

```
            out.println("</html>");
```

```
        }
```

```
}
```



Servlet HelloServlet at /ExampleWebAppProject

Request URI:/ExampleWebAppProject/HelloServlet

Protocol:HTTP/1.1

Path Info:null

A random number:0.66245599935065

doGet() và doPost()

```
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}
```

```
@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}
```

Chạy thử Servlet

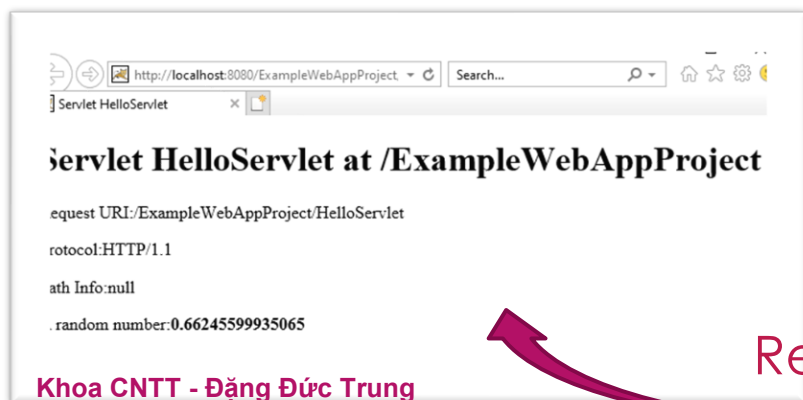
- ▶ Trên cửa sổ quản lý dự án, nhấp phải vào tên file servlet và chọn run file.
- ▶ Quy trình xử lý request:



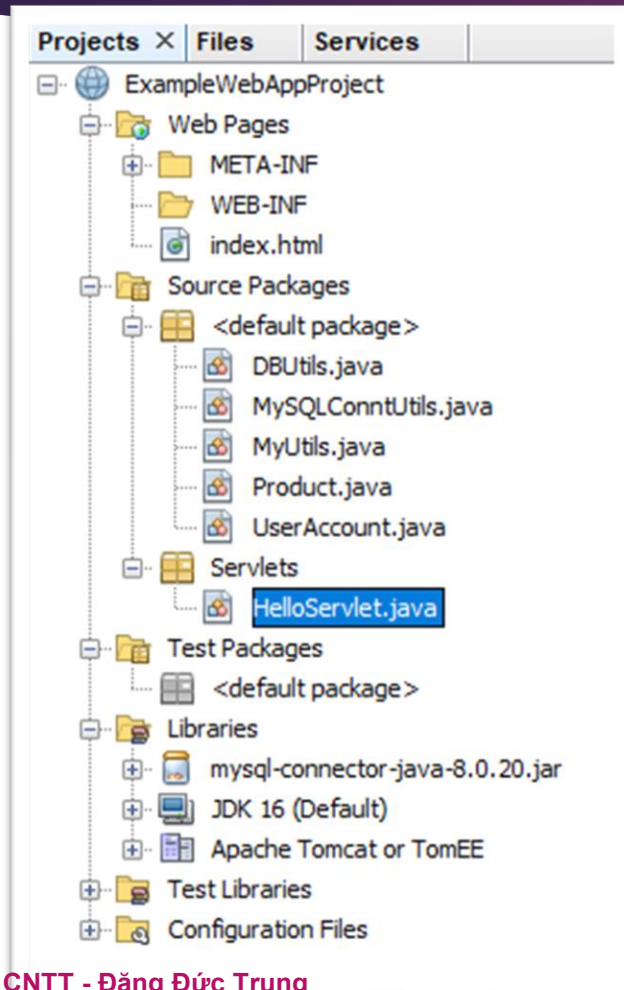
```
@WebServlet(name = "HelloServlet", urlPatterns = {"/HelloServlet"})
public class HelloServlet extends HttpServlet {

    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException { ...19 lines }

    // <editor-fold defaultstate="collapsed" desc="HttpServlet methods. Click on the + sign <
    /** Handles the HTTP <code>GET</code> method ...8 lines */
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        processRequest(request, response);
    }
}
```



Khảo sát cấu trúc tổ chức của Ứng dụng web



- ▶ Source Packages: chứa các file mã nguồn java (*.java)
- ▶ Web Pages: chứa các file html, css, javascript, jsp
- ▶ Libraries: chứa các thư viện
- ▶ Configuration Files: chứa các file cấu hình ứng dụng web.

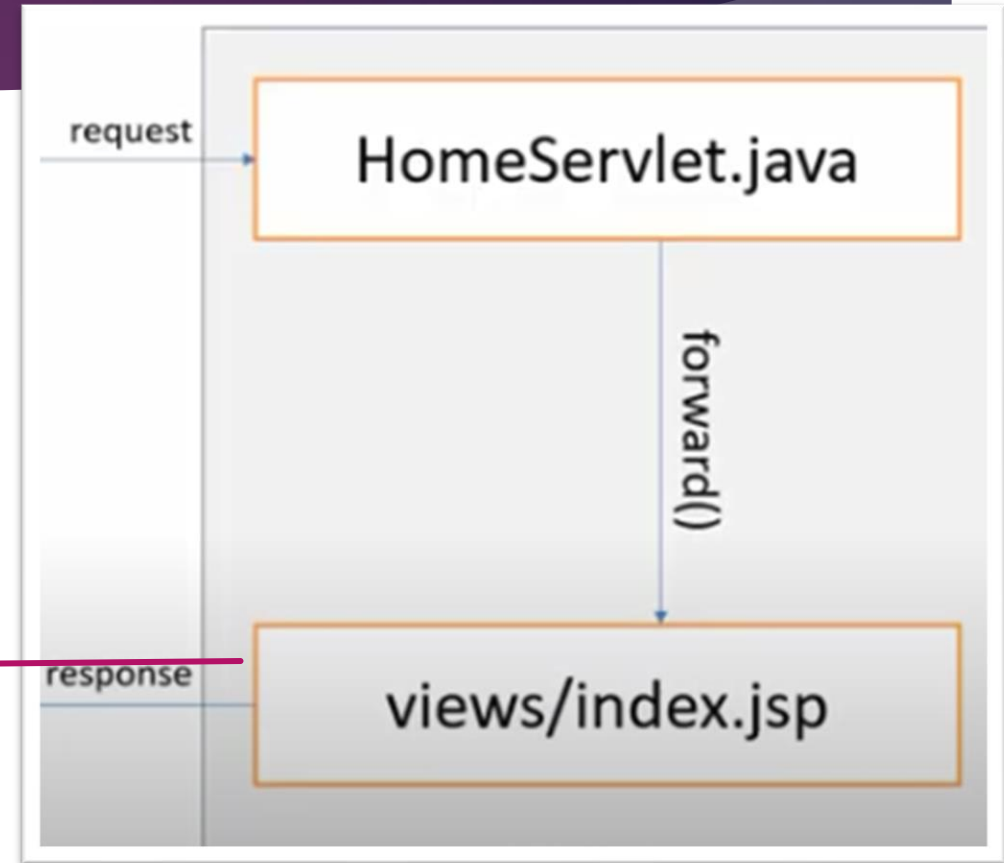
Cấu trúc trang JSP

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>Web App Example</title>
  </head>
  <body>
    <div> WEB APP EXAMPLE</div>
    <div>
      <ul>
        <li><a href="home">Home</a></li>
        <li><a href="login">Login</a></li>
        <li><a href="ProducList">Product List</a></li>
      </ul>
    </div>
  </body>
</html>
```

Quy trình xử lý Request

21

<http://localhost:8080/ExampleWebAppProject/>

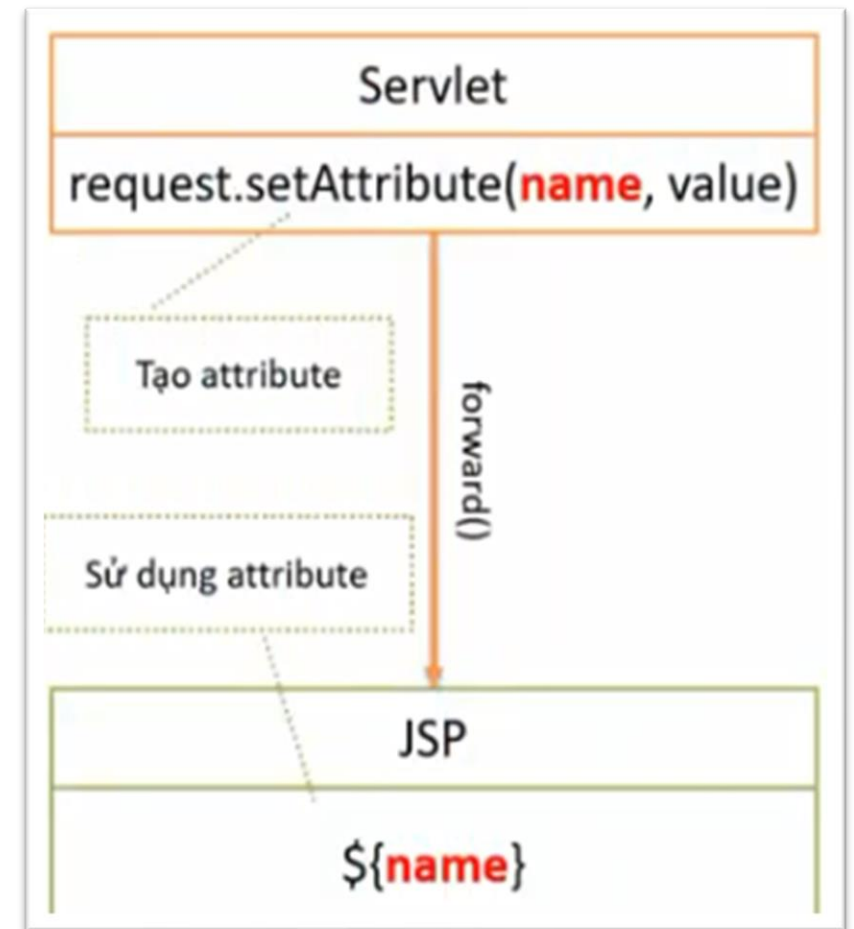


WEB BROWSER

WEB SERVER

TRUYỀN DỮ LIỆU TỪ SERVLET SANG JSP

- ▶ Servlet sẽ nhận và xử lý yêu cầu từ người sử dụng
- ▶ JSP đóng vai trò tạo giao diện để phản hồi người sử dụng
- ▶ JSP cần dữ liệu chia sẻ từ kết quả xử lý của Servlet để sinh giao diện phù hợp
- ▶ Chia sẻ dữ liệu:
 - ▶ Servlet sẽ đặt dữ liệu vào Request trước khi chuyển tiếp sang JSP
 - ▶ JSP lấy và hiển thị dữ liệu



TRUYỀN DỮ LIỆU TỪ SERVLET SANG JSP

23

Servlet:

Name

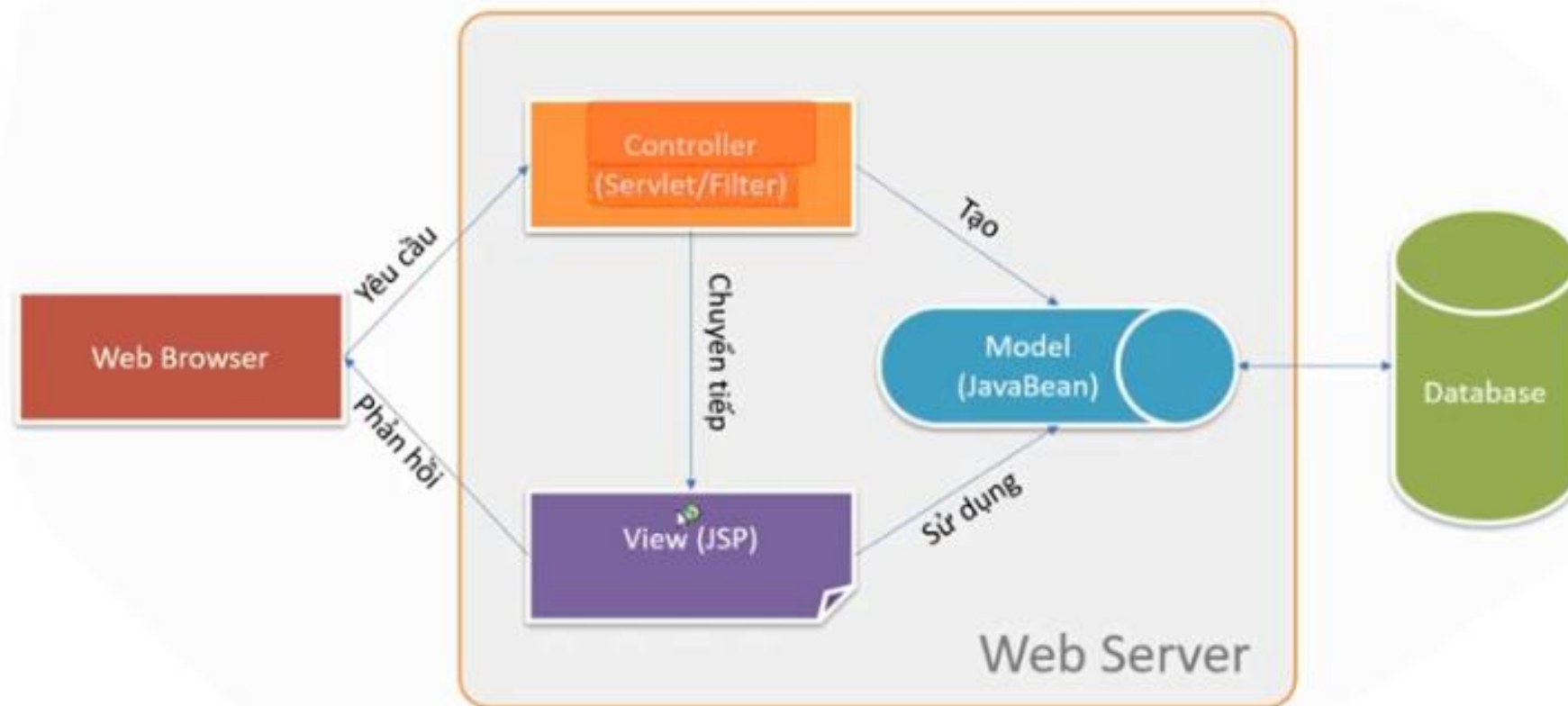
Value

```
request.setAttribute("message", "Chào các bạn 08ĐHCNTT1");
```

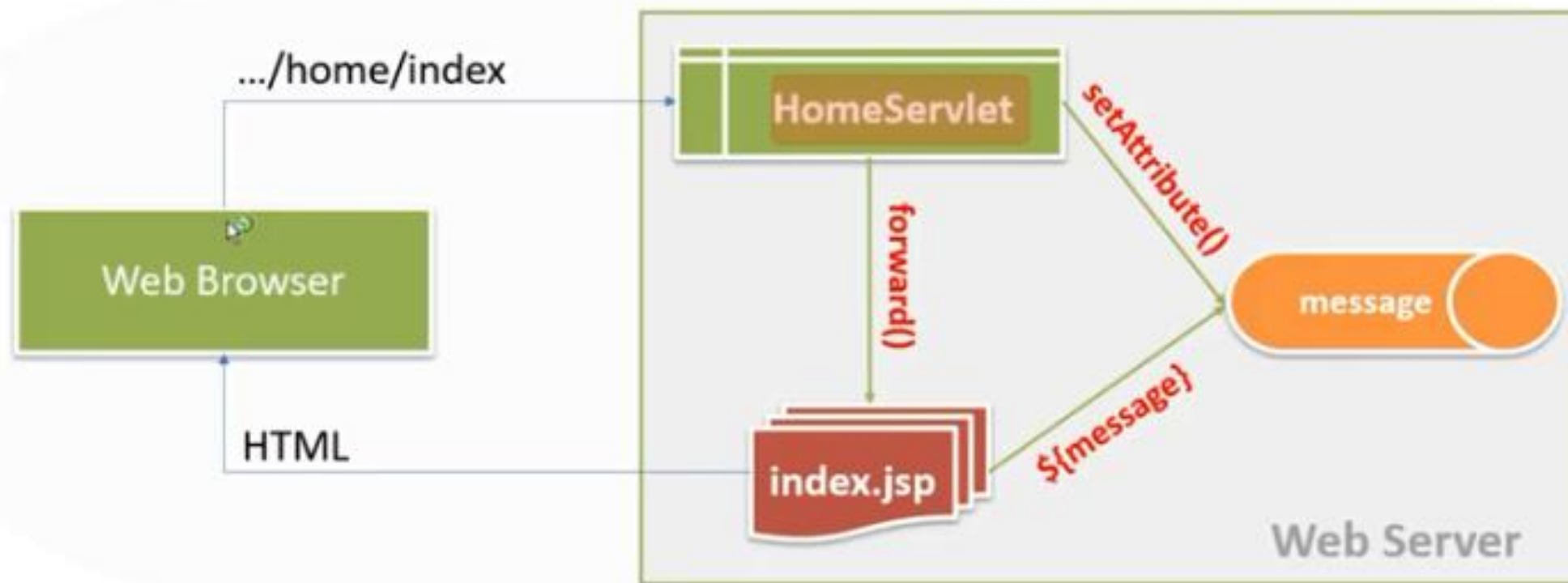
JSP:

`${message}`

Mô hình MVC



Tổ chức ứng dụng web trên MVC



Tham số

- ▶ Tham số (parameter) là dữ liệu gửi đến Servlet từ người dùng thông qua chuỗi truy vấn hoặc form nhập

- ▶ Chuỗi truy vấn:

```
<a href="/index?fullname=Nguyễn Văn A">Liên kết</a>
```

- ▶ Form nhập

```
<form action="/index">
```

```
  <input type="text" name="fullname" value="Nguyễn Văn A"/>
```

```
  <button>Submit</button>
```

```
</form>
```

Đọc và xử lý tham số trên Servlet

- ▶ Để đọc tham số, trong phương thức doGet(request, response) hãy sử dụng lệnh sau:
`String hoten=request.getParameter("fullname");`
- ▶ Với phương thức getParameter() thì kết quả nhận được là giá trị của tham số được gửi đến Servlet hoặc là null nếu tham số không tồn tại.
- ▶ Vì giá trị tham số đọc được luôn luôn là một chuỗi, vì vậy để xử lý có thể phải chuyển đổi sang kiểu phù hợp:
 - ▶ `Integer.parseInt()`
 - ▶ `Double.parseDouble()`
 - ▶ `Boolean.parseBoolean()`

Đóng gói và triển khai

- ▶ Đóng gói ứng dụng web:
 - ▶ Export project thành file kiểu .war
- ▶ Triển khai ứng dụng web
 - ▶ Start Tomcat
 - ▶ Chép sản phẩm đóng gói vào thư mục webapp
- ▶ Truy cập
 - ▶ Mở trình duyệt và nhập địa chỉ trang chủ để chạy

DEMO

- ▶ Trang web tĩnh html
- ▶ JSP/Servlet
- ▶ Gửi dữ liệu lên Server từ người sử dụng thông qua chuỗi truy vấn
- ▶ Gửi dữ liệu lên Server từ người sử dụng thông qua form nhập liệu

2.2. SERVLET VÀ XỬ LÝ FORM

- ▶ Phần 1: Servlet và xử lý form cơ bản
 - ▶ Ánh xạ URL với Servlet
 - ▶ Các phương thức dịch vụ
 - ▶ Xử lý tham số form đơn giản
- ▶ Phần 2: Xử lý form nâng cao
 - ▶ Đọc các tham số nhiều giá trị
 - ▶ Tìm hiểu vòng đời của Servlet và cơ chế đa luồng

Nhắc lại khái niệm Servlet

- ▶ Với công nghệ lập trình web của Sun Microsystem (Java) thì Servlet và JSP là hai thành phần quan trọng
 - ▶ Servlet tiếp nhận yêu cầu, xử lý nghiệp vụ, tương tác CSDL
 - ▶ JSP sinh giao diện (HTML, CSS, Javascript, ...) để phản hồi người sử dụng
- ▶ Về mặt kỹ thuật:
 - ▶ Servlet là một lớp kế thừa từ HttpServlet được cung cấp sẵn
 - ▶ Override phương thức dịch vụ để xử lý yêu cầu người dùng:
 - ▶ doGet(): Xử lý GET request
 - ▶ doPost(): Xử lý POST request
 - ▶ service(): xử lý cả hai loại request

Ví dụ doGet()

32

```
<nl>sign in</nl>
<form method="GET" action="SignInServlet">
<table>
  <tr>
    <td>User Name</td>
    <td><input type="text" name="username"/></td>
  </tr>
  <tr>
    <td>Gender</td>
    <td><input type="text" name="gender"/></td>
  </tr>
  <tr>
    <td>PassWord</td>
    <td><input type="password" name="password"/>
  </tr>
  <tr>
    <td colspan="2"><button>Submit</button><input type="button" value="Reset"/>
  </tr>
</table>
</form>
```



Sign In

User Name

Gender

PassWord



```
@WebServlet(name = "SignInServlet", urlPatterns = {"/SignInServlet"})
public class SignInServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        String username=request.getParameter("username");
        String gender=request.getParameter("gender");
        String password=request.getParameter("password");
    }
}
```

@WebServlet

```
@WebServlet(name = "SignInServlet", urlPatterns = {"/SignInServlet"})
```

- ▶ Annotation @WebServlet được sử dụng để ánh xạ một hoặc nhiều URL với một lớp servlet.
- ▶ Thuộc tính urlPatterns là mảng các URL được ánh xạ.
- ▶ Cú pháp của url:
 - ▶ Phải bắt đầu bởi dấu / hoặc * và phải là duy nhất trong ứng dụng web.
 - ▶ Dấu * đại diện cho một nhóm ký tự bất kỳ.
- ▶ Khi một request có url phù hợp với servlet thì phương thức dịch vụ sẽ thực thi

So sánh POST và GET

34

- ▶ POST và GET là 2 phương pháp truyền dữ liệu từ trình duyệt đến server.
- ▶ Khi submit form với method="GET" thì dữ liệu form được thu thập và tạo thành chuỗi truy vấn (ghép vào sau dấu ? của url) để gửi đến server.
- ▶ Khi submit form với method="POST" thì trình duyệt sẽ tạo một kênh riêng để truyền dữ liệu đến server.
- ▶ GET nhanh hơn POST nhưng có một số hạn chế như:
 - ▶ Dữ liệu nhỏ
 - ▶ Chỉ cho phép text (không upload file được)
 - ▶ Không bảo mật thông tin
- ▶ POST khắc phục được các nhược điểm của GET

Xuất thông tin từ servlet về cho trình duyệt

```
response.setContentType("text/html;charset=UTF-8");  
try (PrintWriter out = response.getWriter()) {  
    out.println("<!DOCTYPE html>");  
    out.println("<html>");  
    out.println("<head>");  
    out.println("<title>Servlet NewServlet</title>");  
    out.println("</head>");  
    out.println("<body>");  
    out.println("<h1>Servlet NewServlet at " + request.getContextPath() + "</h1>");  
    out.println("</body>");  
    out.println("</html>");  
}
```

LẤY DỮ LIỆU TỪ FORM

36

```
<form method="GET" action="SignInServlet">
<table>
  <tr>
    <td>User Name</td>
    <td><input type="text" name="username"/></td>
  </tr>
  <tr>
    <td>Gender</td>
    <td><input type="text" name="gender"/></td>
  </tr>
  <tr>
    <td>PassWord</td>
    <td><input type="password" name="password"/></td>
  </tr>
  <tr>
    <td colspan="2"><button>Submit</button><input type="reset"/></td>
  </tr>
</table>
</form>
```

```
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    String username=request.getParameter("username");
    String gender=request.getParameter("gender");
    String password=request.getParameter("password");
}
```


XỬ LÝ MỘT SỐ TRƯỜNG HỢP ĐẶC BIỆT

- ▶ Với `<select name="cbo_">`
 - ▶ Lấy giá trị mục chọn
- ▶ Với `<input type="radio" name = "rdo_">`
 - ▶ Lấy giá trị radio được chọn
- ▶ Với `<input type="checkbox" name="chk_">`
 - ▶ Lấy giá trị khi có check, null khi không check
- ▶ Với `<button type="submit" name="btn_">`
 - ▶ Lấy giá trị nút bị click, null khi không click



The image shows a screenshot of a web form with several input fields. At the top is a dropdown menu labeled 'United States' with a downward arrow. Below it are two radio buttons: 'Male' (unselected) and 'Female' (selected with a blue dot). Underneath the radio buttons is a checkbox labeled 'Single?' which is checked with a blue square. At the bottom of the form are three buttons: 'Create', 'Update', and 'Delete'.

- ▶ Khai báo các biến đối tượng JavaBeans và truy xuất các thuộc tính và phương thức theo đúng cú pháp của Java

`UserAccount usr=new UserAccount(username,password)`

- ▶ Làm việc với CSDL sử dụng các đối tượng kết nối CSDL: Connection, Statement, ResultSet
- ▶ Cần khai báo import java.sql.*

Làm việc với Database - Connection

- Khai báo và khởi tạo đối tượng Connection:

`Connection conn;`

`conn = MySQLConntUtils.getMySQLConnection();`

trong đó phương thức `MySQLConntUtils.getMySQLConnection()` được thiết kế như sau:

```
public class MySQLConntUtils {  
  
    public static Connection getMySQLConnection() throws ClassNotFoundException, SQLException{  
        String hostname="localhost";  
        String dbName="mytest";  
        String username="root";  
        String password="";  
        return getMySQLConnection(hostname,dbName,username, password);  
    }  
  
    private static Connection getMySQLConnection(String hostname, String dbName, String username, String password)  
        throws ClassNotFoundException, SQLException {  
        Class.forName("com.mysql.jdbc.Driver");  
        String connectionURL="jdbc:mysql://" +hostname+":3306/" +dbName;  
        Connection conn=DriverManager.getConnection(connectionURL, username, password);  
        return conn;  
    }  
}
```

Làm việc với Database - Statement

- ▶ Khai báo và khởi tạo đối tượng Statement:

```
PreparedStatement pstmt = conn.prepareStatement(sql);
```

Trong đó sql là một biến chuỗi chứa chuỗi truy vấn sql

- ▶ Đóng kết nối: `con.close();`
- ▶ Giải phóng kết nối: `con.dispose();`
- ▶ Đóng đối tượng Statement: `pstmt.close();`
- ▶ Giải phóng đối tượng Statement: `pstmt.dispose();`

Làm việc với Database - ResultSet

- ▶ Đối tượng này nắm giữ một tập dữ liệu cho phép bạn thao tác với tập dữ liệu đó bằng các phương thức và thuộc tính của nó.
- ▶ Khai báo: `ResultSet rs = st.executeQuery(sql);`
- ▶ Đọc dữ liệu từ đối tượng: `rs.next()`
- ▶ Kiểm tra tồn tại đối tượng:
`if(rs.next()) { }`
- ▶ Duyệt từng mẫu tin:
`while(rs.next()) { }`
- ▶ Đóng đối tượng ResultSet: `rst.close();`
- ▶ Giải phóng đối tượng ResultSet: `rst.dispose();`

Hết chương 2